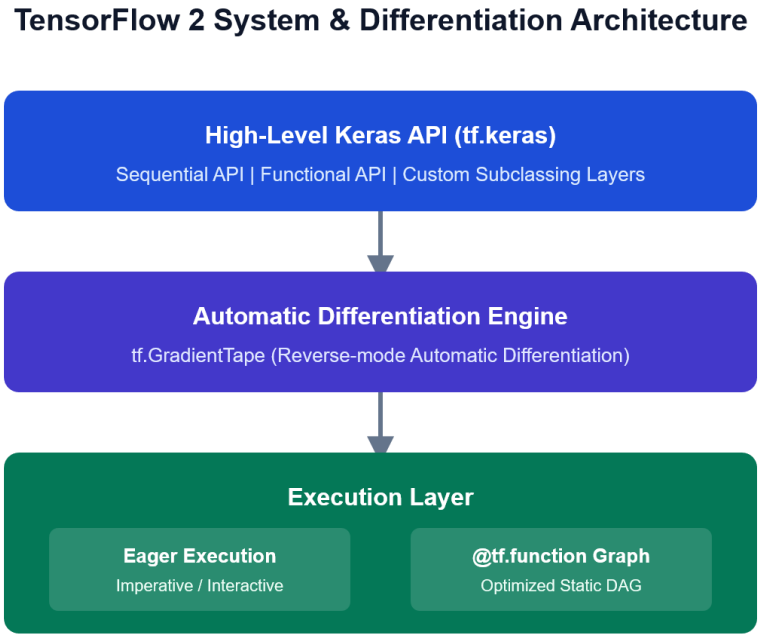
Unit-I: Introduction to Generative AI

1. Overview of Python and TensorFlow 2

Generative Artificial Intelligence focuses on training deep neural network architectures to learn the underlying probability density function $P_{\text{data}}(X)$ of a given training dataset X, enabling the network to sample completely new, synthetic data points $\tilde{x} \sim P_{\text{model}}(X)$. Implementing these complex probabilistic models requires high-performance mathematical computing environments.



A. The Role of Python in Generative AI

Python has emerged as the standard programming language for deep learning and generative modelling due to its clear syntax and robust scientific library ecosystem:

- **NumPy (Numerical Python):**
  - Handles N-dimensional tensor array operations, vector arithmetic, matrix multiplications, and linear algebra.
  - Crucial for constructing random latent noise vectors $z \sim \mathcal{N}(0, \mathrm{I})$ or $z \sim \mathcal{U}(-1,1)$ to feed into generative pipelines.
- **Pandas (Python Data Analysis Library):**
  - Manages structured tabular data, dataset indexing, CSV metadata loading, sequence alignment, and cleaning text/audio metadata.
- **Matplotlib & Seaborn:**
  - Used for diagnostic visualization of training dynamics.
  - Essential for plotting Generator versus Discriminator loss curves over epochs, visualizing feature distribution histograms, and rendering arrays of synthetic images or audio spectrograms.

B. TensorFlow 2 Architecture & Key Components

TensorFlow 2 is an end-to-end, open-source machine learning platform designed by Google. It provides high-level user abstractions alongside low-level execution control:

- **High-Level tf.keras Integration:**

TensorFlow 2 adopts tf.keras as its core high-level object-oriented API. It allows developers to construct neural networks using three distinct paradigms:

- Sequential API: For straightforward stack-of-layers models.
- Functional API: For complex directed acyclic graphs (DAGs) with multiple inputs/outputs (e.g., Conditional GANs).
- Model Subclassing: For complete imperative control over custom forward pass execution.

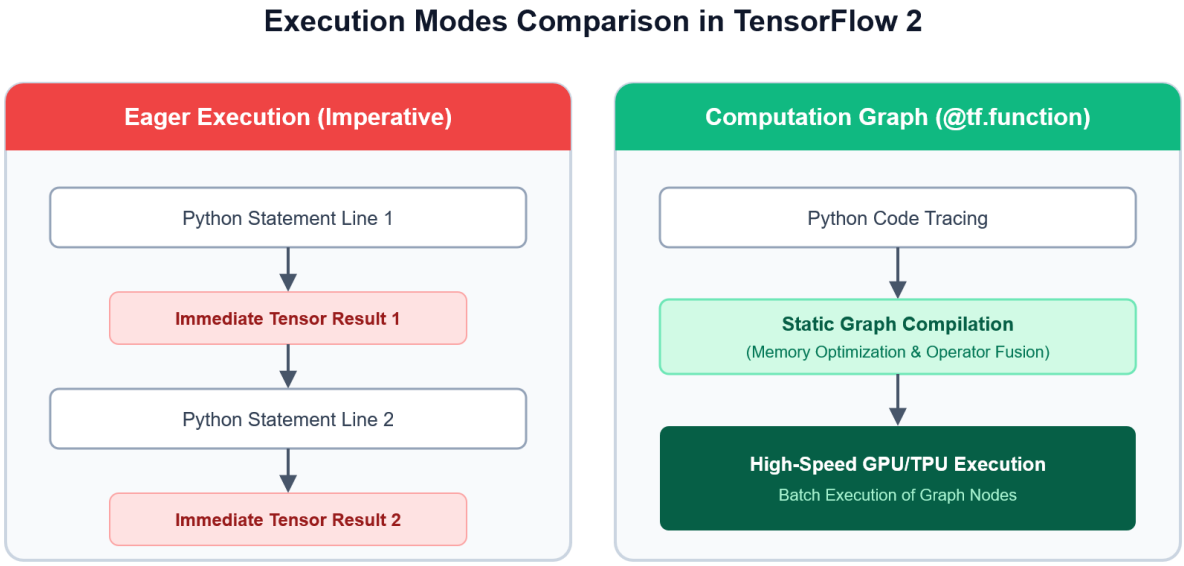
- **Automatic Differentiation via tf.GradientTape:**

Unlike traditional supervised classification models trained using simple model.fit() loops, generative architectures (like GANs) require multi-network adversarial optimization where Generator and Discriminator weights must be updated separately using custom loss functions. tf.GradientTape provides low-level control by recording operations executed on watched tensors within its context block, evaluating gradients using reverse-mode automatic differentiation:

$$\nabla_\theta \mathcal{L} = \frac{\partial \mathcal{L}}{\partial \theta}$$

2. Computation Graph vs. Eager Execution

TensorFlow 2 supports two execution paradigms: Eager Execution (imperative) and Computation Graph Compilation (@tf.function symbolic)



A. Eager Execution (Default Mode in TF2)

- Definition: An imperative, evaluate-immediately programming environment where operations return concrete numeric Tensor values instantly as they are invoked from Python, without building a symbolic graph first.
- Advantages:
  - Debugging Simplicity: Enables native Python control flows and debuggers (e.g., pdb, native print() calls, conditional statements).
  - Fast Prototyping: Allows rapid experimentation and inspection of dynamic array shapes during model development.
- Disadvantages: High runtime overhead due to frequent calls between the Python interpreter and the underlying C++ backend.

B. Computation Graph (@tf.function)

- Definition: A symbolic execution mode where TensorFlow traces Python functions into a high-performance, language-agnostic Dataflow Computation Graph (a Directed Acyclic Graph where nodes represent operations and edges represent Tensors).
- Traces and AutoGraph: Wrapping a Python function with the @tf.function decorator automatically invokes TensorFlow's AutoGraph engine. AutoGraph converts native

Python dynamic constructs (if, while, for, break) into equivalent TensorFlow symbolic graph primitives (tf.cond, tf.while_loop).

- **Advantages:**
  - Performance Optimization: Performs kernel fusion (combining multiple operations into a single GPU call), memory allocation optimization, and constant folding.
  - Parallel Execution: Identifies independent sub-graphs to run concurrently across multiple GPU/TPU threads.
  - Portability & Deployment: Compiled graph assets can be exported via SavedModel format and deployed in C++, Java, iOS, Android, or web environments without requiring a Python interpreter.

Detailed Comparative Analysis

Evaluation Parameter	**Eager Execution Mode**	**Computation Graph Mode (@tf.function)**
Execution Paradigm	Imperative; evaluates ops immediately line-by-line.	Symbolic; builds a static computational graph prior to execution.
Debugging Complexity	Extremely low; compatible with standard Python debuggers (pdb).	Higher; internal graph nodes are abstracted from standard Python inspection.
Execution Velocity	Slower due to Python interpreter overhead per loop iteration.	High-speed; enables kernel fusion and GPU/TPU device optimization.
Control Flow Parsing	Native Python dynamic control structures (if, for).	Converts dynamic statements via AutoGraph into tf.cond & tf.while_loop.
Production Readiness	Suitable for research, inspection, and rapid prototyping.	Suitable for scalable model training and production deployment.

3. Preprocessing and Cleaning Data for Generative AI Applications

Generative networks learn to mimic complex, multi-modal probability distributions $P_{\text{data}}(X)$. Unnormalized, noisy, or unaligned data distorts the latent representation space, leading to training instabilities or complete model failure.

A. Image Preprocessing and Cleaning

- **Pixel Intensity Range Normalization:**

The generator's output layer typically employs non-linear activation functions such as Hyperbolic Tangent (tanh), which outputs values strictly bounded in $[-1,1]$. Input images must be scaled to match this range:

$$\mathrm{X}_{\text{normalized}} = \frac{\mathrm{X} - 127.5}{127.5}$$

(If using sigmoid activations, scaling maps inputs to $[0,1]$ via $\mathrm{X}_{\text{normalized}} = \frac{\mathrm{X}}{255.0}$).

- **Spatial Alignment & Standardization:**

Images from heterogeneous sources must be cropped and resized to uniform spatial dimensions ($H \times W \times C$, e.g., $64 \times 64 \times 3$ or $256 \times 256 \times 3$) using bilinear or bicubic interpolation algorithms to maintain consistent input tensor dimensions.

- **Data Pipeline Optimization:**

Utilizing tf.data.Dataset pipelines to chain asynchronous prefetching (prefetch(tf.data.AUTOTUNE)), batching (batch()), and parallel CPU mapping (map()) ensures GPU compute engines never stall waiting for data loading.

B. Text Preprocessing and Cleaning

- **Subword Tokenization:**

Splitting text strings into subword tokens using algorithms like Byte-Pair Encoding (BPE) or WordPiece. Subword tokenization bounds vocabulary size while ensuring the model handles out-of-vocabulary (OOV) tokens gracefully.

- **Padding and Truncation:**

Variable-length sentences are standardizing to a fixed sequence length L:

  - Sequences longer than L are truncated.
  - Sequences shorter than L are padded using special <PAD> tokens.

- **Numerical Indexing:**

Converting discrete subword tokens into unique numerical integer IDs based on a fixed lookup vocabulary table.

C. Audio and Music Data Preprocessing

- **Piano-Roll Representation:**

Quantizing continuous MIDI files or acoustic musical tracks into 2D/3D binary sparse tensor matrices:

$$\mathrm{M} \in \{0,1\}^{T \times P}$$

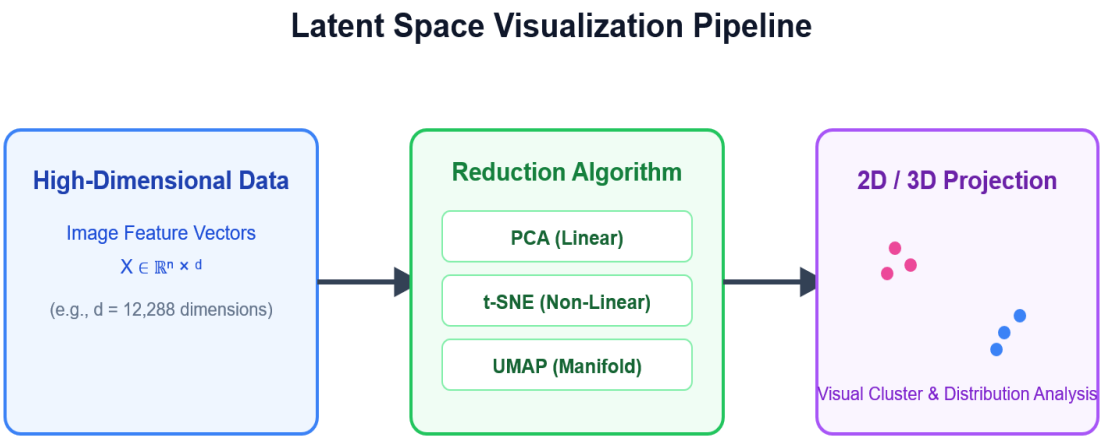
where T represents discrete time steps (e.g., 16th-note ticks) and P represents active MIDI pitch indices (0 to 127).

- **Spectrogram Generation:**

Converting 1D continuous audio time-series waveforms into 2D time-frequency log-magnitude representations using Short-Time Fourier Transforms (STFT).

4. Visualizing Data Distributions and Patterns in Generative AI Datasets

Generative model training requires continuous inspection of high-dimensional feature distributions to detect failure modes like mode collapse (where the generator produces limited variations) or distribution coverage gaps.



A. Dimensionality Reduction Algorithms

Real-world generative datasets exist in very high-dimensional feature spaces (e.g., a small $64 \times 64 \times 3$ RGB image has $12{,}288$ dimensions). Dimensionality reduction techniques project high-dimensional data points down to 2D or 3D coordinate space for visual inspection:

- **Principal Component Analysis (PCA):**
  - Mechanism: A linear dimensionality reduction technique that computes eigenvectors of the data covariance matrix to identify orthogonal axes (principal components) maximizing variance.
  - Use Case: Provides a fast global view of dataset variance, though it fails to capture complex non-linear manifolds.
- **t-Distributed Stochastic Neighbor Embedding (t-SNE):**
  - Mechanism: A non-linear algorithm that converts Euclidean distances between high-dimensional data points into conditional probabilities, minimizing the Kullback-Leibler (KL) divergence between high-dimensional and low-dimensional probability distributions.
  - Use Case: Excellent for visualizing localized clusters in latent spaces.
- **Uniform Manifold Approximation and Projection (UMAP):**
  - Mechanism: A non-linear manifold learning technique built on Riemannian geometry and algebraic topology.
  - Use Case: Preserves both local cluster relationships and overall global data structure, offering faster computation than t-SNE on large datasets.

B. Diagnostic Visual Inspection Methods

- **Latent Space Interpolation Trajectories:**

Linearly or spherically interpolating between two random latent vectors $z_1, z_2 \sim \mathcal{N}(0, \mathrm{I})$ via:

$$z(\alpha) = (1 - \alpha)z_1 + \alpha z_2, \quad \text{where } \alpha \in [0,1]$$

Passing interpolated vectors $z(\alpha)$ through the decoder/generator verifies whether output transformations are smooth and continuous, confirming an organized latent manifold without coverage gaps.

- **Real vs. Synthetic Feature Distribution Overlays:**

Plotting overlapping feature histograms (e.g., mean pixel intensities, sentence length distributions, or pitch frequencies) comparing real dataset samples against generated synthetic samples to evaluate distribution match.